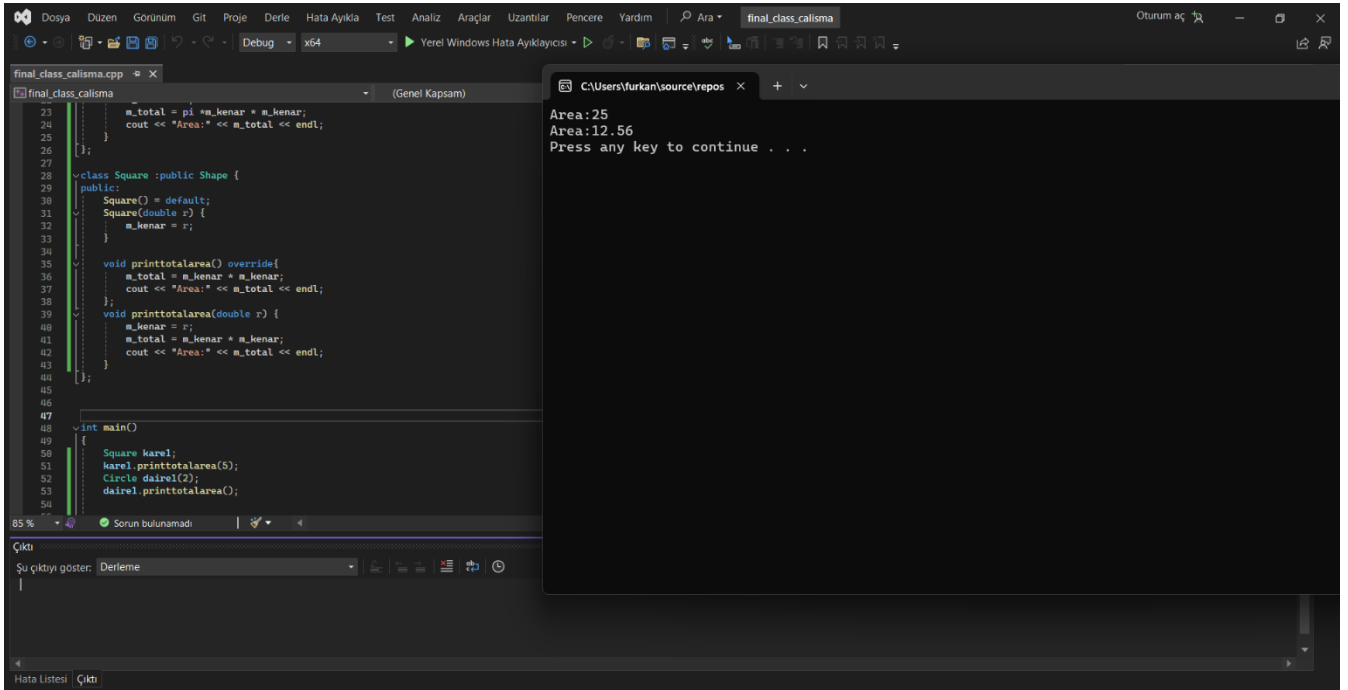


# 23103091042

## Furkan ERTÜRK

Kodum:



The screenshot shows a C++ IDE with the file `final_class_calisma.cpp` open. The code defines a `Shape` class with a `printtotalarea()` method and a `Circle` class that inherits from `Shape`. The `main` function creates a `Circle` object and calls `printtotalarea()`. The output window shows the results: `Area:25` and `Area:12.56`.

```
#include <iostream>
#define pi 3.14
using namespace std;
class Shape {
protected:
    double m_kenar = 0;
    double m_total = 0;
    Shape() = default;
    virtual void printtotalarea()=0;
};
class Circle :public Shape {
public:
    Circle() = default;
    Circle(double r) {
        m_kenar = r;
    }
    void printtotalarea()override {
        m_total = pi * m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
    void printtotalarea(double r) {
        m_kenar = r;
        m_total = pi * m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
};

int main()
{
    Circle kare1;
    kare1.printtotalarea(5);
    Circle daire1(2);
    daire1.printtotalarea();
}
```

Output:

```
Area:25
Area:12.56
Press any key to continue . . .
```

```
#include <iostream>
#define pi 3.14
using namespace std;
class Shape {
protected:
    double m_kenar = 0;
    double m_total = 0;
    Shape() = default;
    virtual void printtotalarea()=0;
};
class Circle :public Shape {
public:
    Circle() = default;
    Circle(double r) {
        m_kenar = r;
    }
    void printtotalarea()override {
        m_total = pi * m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
    void printtotalarea(double r) {
        m_kenar = r;
        m_total = pi * m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
};
```

```

    }
};

class Square :public Shape {
public:
    Square() = default;
    Square(double r) {
        m_kenar = r;
    }

    void printtotalarea() override{
        m_total = m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    };
    void printtotalarea(double r) {
        m_kenar = r;
        m_total = m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
};

int main()
{
    Square kare1;
    kare1.printtotalarea(5);
    Circle daire1(2);
    daire1.printtotalarea();

    system("PAUSE");
    return 0;
}

```

Hocam ödevde nesneleri silin demişsiniz ancak derleyici bunu otomatik olarak yaptığı için gerek duymadım ki zaten class yapısını kullanmamızın da amacı bu tarz işleri derleyiciye bırakmak.

Ama ödevdeki yazan her şeye birebir uygun olsun diye düzenlediğim hali:  
 Kodda sadece main kısmına **pointer ekledim ve delete ile sildim.**

```

#include <iostream>
#define pi 3.14
using namespace std;
class Shape {
protected:
    double m_kenar = 0;
    double m_total = 0;
    Shape() = default;
    virtual void printtotalarea()=0;
};
class Circle :public Shape {
public:
    Circle() = default;
    Circle(double r) {
        m_kenar = r;
    }
    void printtotalarea()override {
        m_total = pi * m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
    void printtotalarea(double r) {
        m_kenar = r;
        m_total = pi *m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
};

class Square :public Shape {
public:
    Square() = default;
    Square(double r) {
        m_kenar = r;
    }

    void printtotalarea() override{
        m_total = m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    };
    void printtotalarea(double r) {
        m_kenar = r;
        m_total = m_kenar * m_kenar;
        cout << "Area:" << m_total << endl;
    }
};

int main()
{
    Square* ptr[1] = { nullptr };
    ptr[0] = new Square;
    ptr[0]->printtotalarea(5);
    delete ptr[0];

    Circle* ptr2[1] = { nullptr };
    ptr2[0] = new Circle();
    ptr2[0]->printtotalarea(2);
    delete ptr2[0];

    system("PAUSE");
    return 0;
}

```